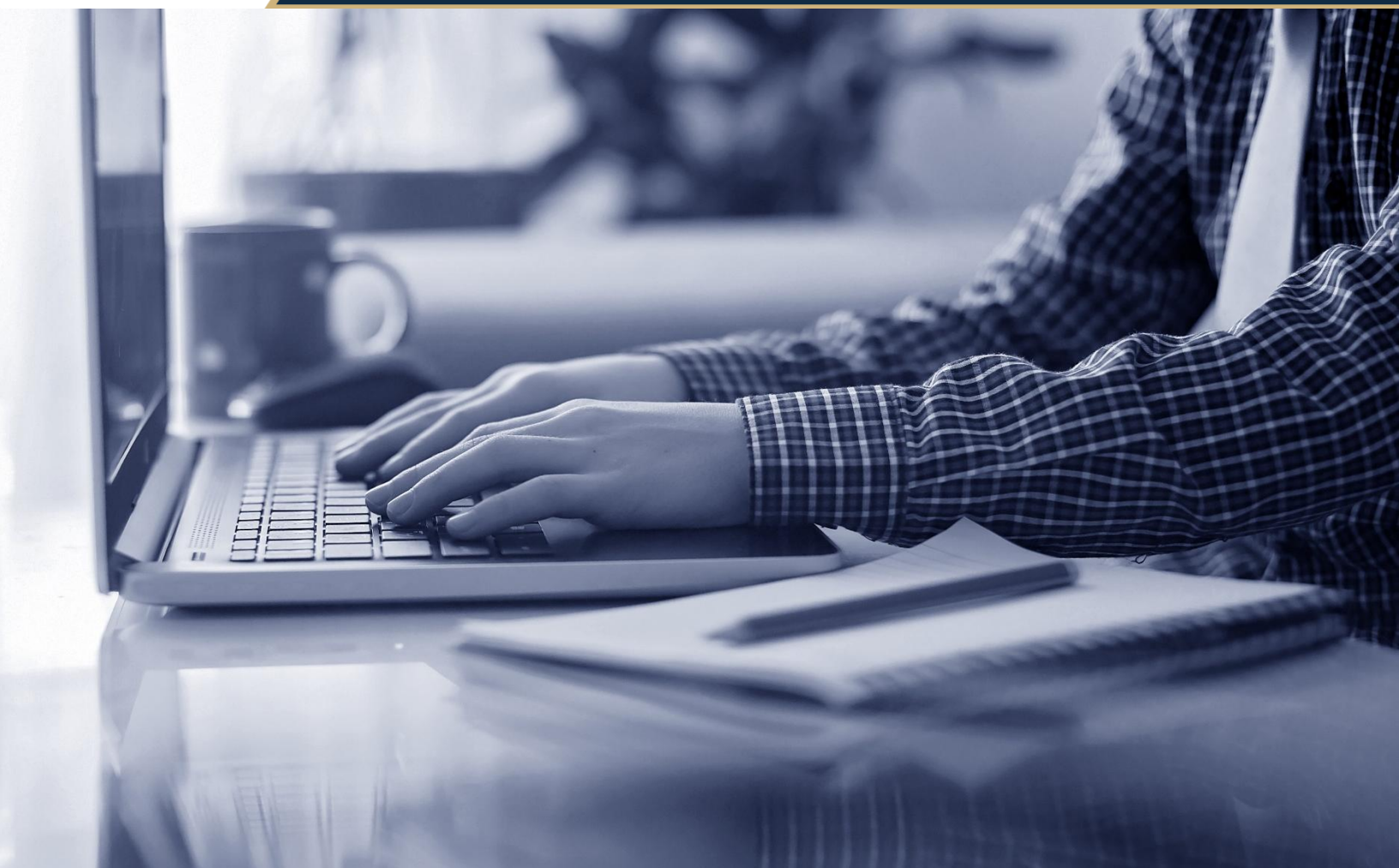




PROGRAMACIÓN PARA LA CIENCIA DE DATOS

CD106ICDA

Apunte N° 1

Fundamentos de programación y manipulación de datos

Introducción

El presente documento tiene como finalidad presentar al y la estudiante una guía clara y aplicada para iniciarse en la programación con Python orientada a la ciencia de datos, abordando desde la sintaxis básica y los tipos/estructuras de datos nativas, hasta el uso introductorio de librerías especializadas como NumPy y pandas para la carga, exploración y manipulación de datos tabulares. Además, se desarrolla el uso de estructuras de control (condicionales y bucles), la definición de funciones para modularizar tareas y el manejo básico de archivos (por ejemplo, lectura/escritura de CSV), integrando estos elementos en pequeños scripts coherentes que producen salidas interpretables (tablas y gráficos con matplotlib).

Asimismo, este apunte activa el aprendizaje mediante ejemplos contextualizados a realidades chilenas (regiones, comunas, indicadores simples, series de tiempo), de modo que cada nuevo concepto se conecte con el anterior y se refuerce tanto de forma conceptual como práctica. La progresión y los ejercicios propuestos están alineados con el resultado de aprendizaje de la unidad, garantizando consistencia entre contenidos, actividades y evidencias.

Junto a lo anterior, verán, a lo largo del documento, algunas preguntas de análisis para corroborar la comprensión de los contenidos presentados, las cuales son útiles para transferir estos conocimientos al ambiente laboral (por ejemplo, criterios de limpieza y consistencia de datos, selección de estructuras adecuadas y decisiones de visualización).

1. PRINCIPIOS BÁSICOS DE LA PROGRAMACIÓN EN PYTHON

1. Principios básicos de programación en Python

En esta primera sección repasaremos los conceptos fundamentales de la programación en Python, sentando las bases para trabajar con datos más adelante. Es probable que hayas visto nociones básicas de programación en cursos previos (por ejemplo, algoritmos o pseudocódigo); aquí las aplicaremos utilizando Python. Comenzaremos revisando la sintaxis básica del lenguaje, los tipos de datos primitivos (numéricos, cadenas de texto y booleanos) y las estructuras de datos nativas que ofrece Python (listas, tuplas y diccionarios). Estos elementos básicos nos permitirán representar información del mundo real (como números de sensores, nombres de ciudades chilenas, valores de verdad en decisiones lógicas, etc.) de forma que la computadora pueda procesarlos.

1.1 Sintaxis básica de Python

Python es un lenguaje de programación de alto nivel conocido por su sintaxis clara y legible. A diferencia de otros lenguajes (como C, Java, etc.), Python utiliza indentación (espacios o tabulaciones al inicio de la línea) para definir bloques de código en lugar de llaves {}. Esto significa que la estructura del programa depende de la correcta indentación: todas las líneas con la misma sangría (indentación) pertenecen al mismo bloque lógico. Por ejemplo, en estructuras que veremos más adelante como condicionales o bucles, deberás indentar el código dentro de ellos. Además:

- Las líneas de código en Python suelen terminar con el fin de línea (no se usa punto y coma; al final, salvo casos opcionales).
- Los comentarios se escriben con el símbolo #. Todo lo que sigue a un # en la misma línea es ignorado por el intérprete. Sirven para anotar el código y hacer más entendible su funcionamiento.
- Python es case-sensitive, es decir, distingue entre mayúsculas y minúsculas; por ejemplo, una variable llamada Datos es diferente de una llamada datos.

Veamos un ejemplo simple de código Python que muestra estos aspectos de la sintaxis:

Este es un comentario de ejemplo. La siguiente línea imprime un saludo:

```
print("Hola mundo")
```

En la línea anterior, print es una función incorporada que muestra el texto en pantalla.

En este fragmento, la función `print()` escribe en la consola el texto "Hola mundo". Notemos que el texto está entre comillas; esto indica que es una cadena de caracteres (string). Más adelante profundizaremos en los tipos de datos, pero es importante reconocer desde ya que las cadenas van entre comillas (simples o dobles) y que la función `print` las muestra por pantalla.

Sugerencia de imagen: Diagrama que ilustre la sintaxis básica de Python, por ejemplo, mostrando cómo la indentación agrupa el código en bloques y cómo un comentario es ignorado durante la ejecución.

1.2 Tipos de datos básicos

En Python existen diversos tipos de datos primitivos para representar información simple. Los principales tipos que utilizaremos son:

- **Númericos:** Incluyen enteros (int) y números de punto flotante (float). Los enteros son números sin parte decimal (por ejemplo, 5, 2023), mientras que los float tienen parte decimal (por ejemplo, 3.1416, -0.5). Python maneja automáticamente la precisión y tamaño de estos números (no es necesario especificar el tamaño del entero, por ejemplo).
- **Cadenas de texto:** Tipo str (string). Representan secuencias de caracteres, como palabras o frases, delimitadas por comillas simples o dobles. Ejemplos: "Santiago", 'Hola, ¿cómo estás?'. Las cadenas pueden incluir casi cualquier símbolo (letras, números, espacios, signos) y tienen asociados métodos útiles para manipular texto (como convertir a mayúsculas, cortar subcadenas, etc.).
- **Booleanos:** Tipo bool. Solo tienen dos valores posibles: True (verdadero) o False (falso). Se utilizan en operaciones lógicas y condicionales para representar la verdad o falsedad de una condición. Por ejemplo, podemos tener es_mayor = True para indicar que cierta condición se cumple.

Variables: Para usar estos valores en un programa, normalmente los almacenamos en *variables*. En Python, una variable se crea al asignarle un valor usando el signo =. Python es un lenguaje de *tipado dinámico*, lo que significa que una variable puede cambiar de tipo durante la ejecución, y no es necesario declarar su tipo explícitamente.

Por ejemplo:

Ejemplos de variables con distintos tipos de datos

```
temperatura = 20      # Entero (int) que podría representar grados Celsius
ciudad = "Valdivia"   # Cadena de texto (str) con el nombre de una ciudad
es_lluvioso = False  # Booleano (bool) indicando si está lloviendo o no
precipitacion = 3.5   # Flotante (float) representando mm de lluvia caídos
```

En este ejemplo, temperatura se inicializa con un valor entero 20, ciudad con una cadena "Valdivia", es_lluvioso con un booleano False, y precipitacion con un float 3.5. Python infiere automáticamente el tipo según el valor asignado.

También podemos convertir entre tipos cuando sea necesario (lo que se conoce como *casting*). Por ejemplo, si quisiéramos convertir un número a texto para concatenarlo, usamos la función str(). Inversamente, int() convierte texto numérico en entero, y float() a flotante.

Ejemplo:

```
poblacion_str = "15000"      # Esto es una cadena que contiene un número
poblacion_num = int(poblacion_str) # Ahora población_num es el número entero 15000
print(poblacion_num + 5)     # Imprime 15005, sumando 5 al entero
```

En el snippet anterior, la variable poblacion_str es una cadena "15000". Tras int(poblacion_str), la variable poblacion_num pasa a ser el número 15000 (int). Sumando 5 obtenemos 15005. Este ejemplo muestra

la necesidad de conversión: si hubiéramos intentado sumar `poblacion_str + 5` sin convertir, Python daría error porque no se puede sumar un texto con un número directamente.

Ejemplo: Imaginemos que queremos almacenar la temperatura actual en Santiago y si está pronosticada lluvia. Usaríamos un valor numérico para la temperatura (por ejemplo 28 grados Celsius en verano) y un booleano para la lluvia (False si no va a llover). Además, podríamos guardar el nombre de la ciudad en una cadena. Por ejemplo:

```
ciudad = "Santiago"
temperatura = 28
pronostico_lluvia = False
print("Ciudad:", ciudad)
print("Temperatura esperada:", temperatura, "°C")
if pronostico_lluvia:
    print("Llevar paraguas.")
else:
    print("No se pronostica lluvia.")
```

En este código, definimos `ciudad`, `temperatura` y `pronostico_lluvia`. Luego imprimimos la ciudad y temperatura. La estructura `if/else` (que explicaremos en detalle en la siguiente sección) imprime un mensaje diferente dependiendo de si `pronostico_lluvia` es `True` o `False`. Para *Santiago* con 28°C y `False`, el resultado sería:

```
Ciudad: Santiago
Temperatura esperada: 28 °C
No se pronostica lluvia.
```

Nota: Python permite usar coma en `print` para imprimir múltiples valores seguidos de espacios automáticamente. En el ejemplo, `print("Temperatura esperada:", temperatura, "°C")` concatena la cadena, el número y "°C" separados por espacios.

1.3 Estructuras de datos nativas: listas, tuplas y diccionarios

Los tipos primitivos mencionados (int, float, str, bool) representan valores individuales. Sin embargo, en ciencia de datos a menudo necesitamos manejar colecciones de valores (por ejemplo, una serie de temperaturas diarias, un conjunto de nombres de ciudades, etc.). Python proporciona varias estructuras de datos nativas para agrupar valores:

Listas (list)

Una lista es una secuencia ordenada de elementos, que pueden ser de cualquier tipo. Se definen usando corchetes [], separando los elementos por comas. Las listas en Python son mutables, lo que significa que podemos modificar sus elementos después de creada (agregar, quitar o cambiar valores).

Ejemplos de listas:

Lista de números (por ejemplo, temperaturas máximas diarias en °C durante una semana en Santiago)

```
temperaturas_semana = [30, 32, 29, 27, 26, 28, 31]
```

Lista mixta (puede contener distintos tipos; ej: nombre de ciudad, población en miles, si es capital regional)

```
info_ciudad = ["Arica", 226, True] # "Arica", 226 mil habitantes, True indicando que es capital regional
```

Lista vacía (sin elementos, útil para luego llenarla)

```
datos = []
```

En `temperaturas_semana`, tenemos 7 valores enteros que representan temperaturas diarias. El orden es el mismo en que se definieron: el primer elemento es 30 (lunes, por ejemplo), luego 32 (martes), y así sucesivamente. Podemos acceder a elementos individuales de la lista usando un índice entero entre corchetes []. Importante: en Python (y muchos lenguajes) los índices comienzan en 0. Esto significa que `temperaturas_semana[0]` es 30, `temperaturas_semana[1]` es 32, y `temperaturas_semana[6]` sería 31 (el último elemento, índice 6 porque tenemos 7 elementos).

Algunas operaciones comunes con listas:

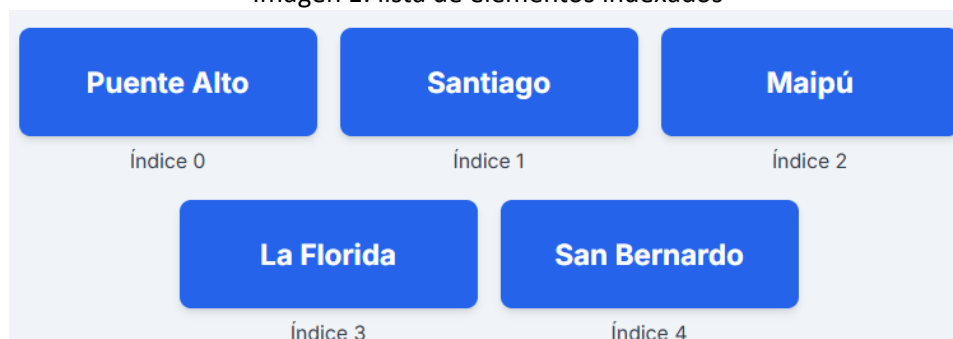
- Agregar elementos: usando el método `append`. Ej: `temperaturas_semana.append(25)` agregaría el valor 25 al final de la lista.
- Modificar elementos: se puede asignar a una posición específica. Ej: `temperaturas_semana[2] = 28` cambiaría el tercer elemento (índice 2) de 29 a 28.
- Recorrer la lista: mediante bucles (lo veremos en detalle en la sección de bucles). Se puede hacer `for t in temperaturas_semana: ...` para iterar por cada temperatura.
- Longitud de la lista: con la función `len(temperaturas_semana)` obtenemos cuántos elementos contiene (en este caso, 7).

Ejemplo: Supongamos que queremos almacenar los nombres de algunas comunas del Gran Santiago que destacaron por su población en el censo. Podríamos usar una lista de cadenas:

```
comunas_mas_pobladas = ["Puente Alto", "Maipú", "Santiago", "La Florida", "Antofagasta"]  
print("Comunas más pobladas:", comunas_mas_pobladas)  
print("La primera comuna de la lista es:", comunas_mas_pobladas[0])
```

Esto imprimiría la lista completa de comunas y luego el primer elemento "Puente Alto". Podemos sugerir una imagen aquí: por ejemplo, una representación visual de una lista como una secuencia de cajas en fila (cada caja con un elemento y su índice correspondiente debajo: 0, 1, 2, ...).

Imagen 1: lista de elementos indexados



Fuente: imagen creada con IA

Tuplas (tuple)

Las tuplas son muy similares a las listas en cuanto a que también son secuencias ordenadas de elementos. La diferencia clave es que las tuplas son inmutables: una vez creada una tupla, sus valores no pueden modificarse (no es posible agregar, quitar o cambiar elementos). Se definen usando paréntesis () en lugar de corchetes.

Ejemplos de tuplas:

```
coordenadas_santiago = (-33.45, -70.66) # Latitud y longitud de Santiago (aprox)  
punto = (5, 9) # Tupla de dos enteros  
valores = (1, 2, 3, 4) # Tupla de enteros  
un_solo = ("Chile",) # Tupla de un solo elemento (nota la coma final necesaria)
```

En el ejemplo, `coordenadas_santiago` es una tupla con dos floats que representan la latitud y longitud. Si intentáramos hacer `coordenadas_santiago[0] = -34`, Python arrojará un error porque no se puede asignar a elementos de una tupla existente. Las tuplas son útiles para representar estructuras fijas de datos donde los valores tienen un significado posicional (por ejemplo, coordenadas (x, y), o registros que no deberían cambiar). También se utilizan implícitamente al retornar múltiples valores de una función.

Puedes acceder a los elementos igual que en listas, usando índices (ej: `punto[1]` daría 9). También soportan `len(punto)` para longitud (aquí 2), pero no tienen métodos como `append` debido a su inmutabilidad.

Ejemplo: Podríamos usar una tupla para representar una fecha (año, mes, día) por simplicidad, aunque existe un tipo específico `datetime` en Python. Por ejemplo, la fecha de independencia de Chile: `fecha_independencia = (1810, 9, 18)` representando 18 de septiembre de 1810 como (año, mes, día). Esta tupla no debería cambiar una vez definida.

Diccionarios (dict)

Los diccionarios son estructuras de datos que almacenan pares clave-valor. En un diccionario, cada elemento (valor) está asociado a una clave única, que actúa como un identificador. Esto permite un acceso rápido al valor a través de su clave, en lugar de por posición como en listas/tuplas. En Python, los diccionarios se definen con llaves `{}` y usan el formato "clave": valor para cada par, separado por comas.

Un diccionario no mantiene un orden garantizado en versiones antiguas de Python, pero desde Python 3.7+ sí mantiene el orden de inserción. Aun así, conceptualmente pensamos en diccionarios como conjuntos de pares desordenados, donde la indexación es por clave, no por posición.

Ejemplos de diccionarios:

Diccionario de ejemplo: población (en miles) de algunas ciudades chilenas

```
poblacion_ciudades = {  
    "Santiago": 6510, # (datos imaginarios, en miles de habitantes)  
    "Valparaiso": 302,  
    "Concepcion": 223,  
    "La Serena": 147  
}
```

Otro ejemplo: códigos de región a nombre de región

```
codigos_region = {  
    13: "Metropolitana",  
    5: "Valparaíso",  
    8: "Biobío",  
    15: "Arica y Parinacota"  
}
```

En `poblacion_ciudades`, las claves son nombres de ciudades (cadenas) y los valores son números que representan sus poblaciones en miles. Por ejemplo, "Santiago" → 6510 significa que Santiago tiene aproximadamente 6.510.000 habitantes (6.51 millones). Podemos acceder a un valor dado su clave: `poblacion_ciudades["Concepcion"]` resultaría en 223. Si intentamos acceder con una clave que no existe, obtendremos un error `KeyError`. Para evitarlo, se puede comprobar primero con el operador `in` (ej: "Antofagasta" `in poblacion_ciudades` devuelve `False` si no está).

En `codigos_region`, las claves son números (los códigos tradicionales de regiones en Chile) y los valores son los nombres correspondientes. Así, `codigos_region[5]` es "Valparaíso".

Operaciones comunes con diccionarios:

- Acceder a valor: `diccionario[clave]` devuelve el valor asociado. Ej: `poblacion_ciudades["La Serena"]` → 147.
- Agregar o modificar valor: `diccionario[nueva_clave] = valor`. Si la clave no existía se añade, si ya existía, se sobrescribe el valor. Ej: `poblacion_ciudades["Antofagasta"] = 425` (agrega la ciudad Antofagasta con población 425 mil).
- Eliminar par: con `del diccionario[clave]` se elimina ese par clave-valor. Ej: `del poblacion_ciudades["La Serena"]` quitaría esa entrada.
- Longitud: `len(diccionario)` da la cantidad de pares almacenados.
- Recorrer diccionario: usando un bucle `for`. Iterar sobre un diccionario recorre sus claves por defecto. Veremos un ejemplo de iteración en la sección de bucles.

Ejemplo: Podríamos tener un diccionario que asocie nombres de regiones de Chile con sus capitales regionales. Por ejemplo:

```
capitales_regionales = {  
    "Antofagasta": "Antofagasta",  
    "Araucanía": "Temuco",  
    "Magallanes": "Punta Arenas",  
    "Metropolitana": "Santiago"  
}  
print("Capital de Araucanía:", capitales_regionales["Araucanía"])
```

Este diccionario guarda cuatro regiones con su capital. Al imprimir la capital de la Araucanía obtenemos *Temuco*.

Sugerencia de imagen: Diagrama de un diccionario mostrando pares clave-valor, quizás como cajas conectadas: cada clave apuntando a su valor. Por ejemplo, una caja con clave "Metropolitana" apuntando a "Santiago", otra con "Araucanía" apuntando a "Temuco", etc., para ilustrar la relación.

2. Estructuras de control de flujo

En cualquier programa, no basta con ejecutar instrucciones secuencialmente; a menudo necesitamos tomar decisiones (por ejemplo, ejecutar cierto código solo si se cumple una condición) o realizar repeticiones (por ejemplo, procesar todos los elementos de una lista uno por uno). Estas necesidades se cubren con las estructuras de control de flujo: las sentencias condicionales (if/else) y los bucles (for y while). A continuación, relacionaremos estos conceptos con situaciones prácticas. Por ejemplo, en ciencia de datos podríamos querer *"si un valor es negativo, reemplazarlo por 0"* (decisión) o *"iterar sobre todas las filas de un dataset para calcular una suma"* (repetición).

2.1 Sentencias condicionales (if/else)

Las sentencias condicionales permiten ejecutar bloques de código solo si se cumple cierta condición. En Python, la sintaxis básica es:

```
if condición:
    # código si la condición es verdadera
elif otra_condición:
    # código si la primera no se cumplió pero esta sí (opcional, puede haber varios elif)
else:
    # código si ninguna de las condiciones anteriores fue verdadera (opcional)
```

La palabra clave if inicia la estructura, seguida de una expresión booleana (la condición). Si esa expresión resulta True, se ejecuta el bloque indentado debajo del if. Si es False, Python salta ese bloque. Podemos encadenar una o más condiciones adicionales usando elif ("else if") con sus propias condiciones. Finalmente, else captura todos los demás casos (cuando ninguna condición anterior fue verdadera).

Algunos operadores lógicos y relacionales útiles en las condiciones:

- Operadores relacionales: == (igual a), != (distinto de), > (mayor que), < (menor que), >= (mayor o igual), <= (menor o igual).

Ejemplo: if x >= 10: comprueba si x es al menos 10.

- Operadores lógicos: and (y lógico), or (o lógico), not (negación lógica).
Ejemplo: if (edad >= 18) and ciudadano: será True solo si ambas condiciones se cumplen (mayor de edad y ciudadano es True).

Ejemplo simple: Determinar si un número es positivo, negativo o cero:

```
numero = -5
if numero > 0:
    print("El número es positivo")
elif numero < 0:
    print("El número es negativo")
else:
    print("El número es cero")
Para numero = -5, la salida sería "El número es negativo".
```

Ejemplo: Supongamos que un sistema de calificaciones a nivel de institutos profesionales indica aprobación o reprobación. En Chile, usualmente la nota mínima aprobatoria es 4.0 (en una escala de 1.0 a 7.0). Podemos usar un `if` para determinar si un estudiante aprobó:

```
nota = 3.7 # nota de un estudiante en cierta asignatura
if nota >= 4.0:
    print("Aprobado")
else:
    print("Reprobado")
```

Dado que la nota es 3.7 (menor a 4.0), el resultado impreso sería "Reprobado". Si cambiamos `nota = 5.5`, entonces la condición del `if` sería verdadera y se imprimiría "Aprobado".

Podemos ampliar el ejemplo para incluir un rango de notas con distinto mensaje. Por ejemplo, clasificar la nota en sobresaliente, aprobado o reprobado:

```
nota = 6.2
if nota < 4.0:
    print("Reprobado")
elif nota >= 6.0:
    print("Aprobado con distinción") # notas 6.0 o más, destacar rendimiento
else:
    print("Aprobado")
```

Con `nota = 6.2`, se cumple la condición del `elif` y veríamos "Aprobado con distinción". Para `nota = 5.0` (ni `<4` ni `>=6`), ninguna de las primeras dos condiciones es verdadera, por lo que caería en el `else` imprimiendo "Aprobado".

Importante: Observa la indentación en estos ejemplos. Cada bloque dependiente de un `if/elif/else` está indentado (por convención se usan 4 espacios). También notemos que la condición va seguida de dos puntos : al final de la línea, indicando el inicio de un bloque indentado.

Las condiciones no tienen por qué ser comparaciones numéricas solamente; podemos evaluar cualquier expresión booleana, incluso variables booleanas directamente. Por ejemplo, `if pronostico_lluvia:` es equivalente a `if pronostico_lluvia == True:` y ejecutará el bloque si `pronostico_lluvia` es `True`. En Python, además, ciertos valores se consideran *True* o *False* implícitamente (ej. 0 se considera *False*, cadenas vacías "" también *False*, listas vacías *False*, etc.), pero para evitar confusión es mejor usar comparaciones explícitas cuando se aprende.

Ejemplo adicional: Imagina que estamos limpiando datos de temperaturas registradas y queremos marcar valores erróneos. Supongamos que la temperatura jamás debería sobrepasar 60°C ni ser inferior a -50°C en nuestro contexto. Podemos usar un `if` para detectar valores fuera de ese rango:

```
temperatura = 75 # valor extremo erróneo
if temperatura < -50 or temperatura > 60:
    print("Temperatura fuera de rango plausible, podría ser un error de dato.")
```

Como `75 > 60`, la condición se cumple y se avisa que el dato es sospechoso. Este tipo de chequeo lógico ayuda a garantizar la consistencia de los datos detectando anomalías, lo cual es parte importante de la ciencia de datos.

2.2 Bucles for

Los bucles for se utilizan para iterar sobre elementos de una secuencia (lista, tupla, cadena, etc.) o cualquier iterable (como un rango de números). En Python, la sintaxis del for es:

```
for variable in secuencia:  
    # bloque de código que usa 'variable'
```

El bucle tomará cada elemento de secuencia (uno por uno, en orden) y lo asignará a variable, ejecutando luego el bloque indentado. Esto se repite hasta recorrer todos los elementos. Es muy útil para procesar conjuntos de datos elemento por elemento.

Ejemplos básicos:

```
# Recorrer una lista de números y calcular su suma  
valores = [5, 3, 8, 4]  
suma = 0  
for v in valores:  
    suma += v # equivalente a suma = suma + v  
print("La suma es:", suma) # Debería mostrar 20  
  
# Iterar sobre una cadena de texto para contar letras  
palabra = "Data"  
for letra in palabra:  
    print(letra)  
# Esto imprimirá D, luego a, luego t, luego a (cada una en nueva línea)
```

En el primer ejemplo, v va tomando los valores 5, luego 3, luego 8, luego 4. Vamos acumulando en suma cada valor. Al final del bucle, la suma es 20. En el segundo ejemplo, iteramos sobre cada carácter de la palabra "Data", imprimiendo uno por línea.

Con for es frecuente usar la función incorporada range() para generar secuencias de números, especialmente cuando se necesita un contador. range(n) genera los enteros desde 0 hasta n-1. Por ejemplo, range(5) produce la secuencia [0,1,2,3,4]. También puede recibir inicios y saltos: range(inicio, fin, paso).

Ejemplo:

```
# Imprimir los números del 1 al 5  
for i in range(1, 6):  
    print(i)  
# Esto imprimirá 1 2 3 4 5 (cada número en línea separada)
```

Aquí usamos range(1,6) que genera 1,2,3,4,5. La variable i toma esos valores sucesivamente.

Iterando estructuras de datos: Podemos usar for para recorrer listas, tuplas e incluso diccionarios:

- Lista/tupla: el for entrega cada elemento en orden.
- Diccionario: al iterar un dict, por defecto se obtienen las claves. Si queremos obtener clave y valor juntos, podemos iterar sobre diccionario.items(), que proporciona tuplas (clave, valor) en cada iteración.

Ejemplo: Tenemos una lista con las precipitaciones (en mm de lluvia) registradas en Concepción durante una semana y queremos calcular el total semanal:

```
precipitaciones = [5, 12, 0, 20, 4, 0, 0] # mm de lluvia de lunes a domingo
total = 0
for mm in precipitaciones:
    total += mm
print("Lluvia total en la semana:", total, "mm")
```

Supongamos que los valores corresponden a la primera semana de julio y que en Concepción llovió 5 mm el lunes, 12 mm el martes, etc., hasta el domingo con 0 mm. El código sumará todos esos valores y mostrará el total (en este caso 41 mm). Esto ilustra un uso práctico del for: recorrer datos meteorológicos para obtener un resultado acumulado.

Iteración sobre un diccionario (ejemplo adicional): Usando el diccionario poblacion_ciudades definido antes, podemos iterar así:

```
for ciudad, poblacion in poblacion_ciudades.items():
    print(ciudad, "tiene", poblacion, "mil habitantes")
```

Esto iterará entregando tuplas clave-valor; en cada vuelta, ciudad será la clave (nombre de la ciudad) y poblacion el valor asociado. La salida podría ser:

- Santiago tiene 6510 mil habitantes
- Valparaíso tiene 302 mil habitantes
- Concepción tiene 223 mil habitantes
- La Serena tiene 147 mil habitantes

Control de bucle con break/continue: Python ofrece dos sentencias útiles dentro de bucles:

- break: termina el bucle inmediatamente, saliendo de él.
- continue: salta a la siguiente iteración (omite el resto del bloque actual para esa iteración).

Por ejemplo, si buscamos un elemento en una lista, podemos usar break cuando lo encontramos para no seguir iterando inútilmente:

```
buscado = "Maipú"
for comuna in comunas_mas_pobladas:
    if comuna == buscado:
        print("Encontrada:", comuna)
        break # salimos del for al encontrarla
```

Si buscado es "Maipú", el loop imprimirá "Encontrada: Maipú" y luego terminará antes de recorrer las comunas restantes. Por otro lado, continue podría usarse para saltar ciertos casos. Imagina que queremos contar, en la lista de precipitaciones, solo los días con lluvia (>0):

```
dias_lluviosos = 0
for mm in precipitaciones:
    if mm == 0:
        continue # saltar días sin lluvia
    dias_lluviosos += 1
print("Días con lluvia en la semana:", dias_lluviosos)
```

Aquí cada vez que mm es 0, hacemos continue, que hace que el bucle pase inmediatamente al próximo día sin ejecutar días_lluviosos += 1 en ese caso. De este modo solo contamos días con precipitación. Con la lista [5,12,0,20,4,0,0], contaría 4 días con lluvia.

2.3 Bucles while

El bucle while repite un bloque de código mientras se cumpla una condición booleana. Su sintaxis es:

```
while condición:  
    # código a ejecutar repetidamente mientras la condición sea True
```

A diferencia del for (que itera un número conocido de veces sobre una secuencia), el while se utiliza cuando no sabemos de antemano cuántas iteraciones serán necesarias, o queremos repetir hasta que ocurra algo. ¡Cuidado!: Es fundamental que en algún momento dentro del bucle while la condición llegue a ser falsa, de lo contrario el bucle sería infinito. Normalmente se actualiza alguna variable dentro del bucle para acercarse al fin.

Ejemplo básico: Imprimir los números del 1 al 5 usando while:

```
contador = 1  
while contador <= 5:  
    print(contador)  
    contador += 1 # incrementar el contador en 1
```

En este ejemplo inicializamos contador en 1. La condición del while verifica contador <= 5. Mientras eso sea verdadero, ejecuta el bloque: imprime el valor actual y luego lo incrementa. Cuando contador llega a 6, la condición contador <= 5 pasa a False y el bucle termina. La salida será igual que con el for anterior: 1, 2, 3, 4, 5 en líneas separadas.

Ejemplo: Supongamos que queremos encontrar cuántos años toma para que una inversión crezca por encima de cierto monto con interés compuesto. Imagina que un fondo de investigación en Chile comienza con 1 millón de pesos y crece un 5% anual; queremos saber en cuántos años superará los 2 millones:

```
fondos = 1000000 # 1,000,000 CLP  
anos = 0  
while fondos < 2000000:  
    fondos *= 1.05 # incrementar fondos en 5%  
    anos += 1  
print("Años necesarios para duplicar los fondos:", anos)
```

Aquí fondos se va actualizando cada año multiplicándolo por 1.05. El while continúa hasta que fondos < 2000000 sea False, es decir, hasta que los fondos alcancen o superen los 2,000,000. El resultado impreso sería el número de años necesarios (en este caso alrededor de 15 años, ya que $1,000,000 * 1.05^{15} \approx 2,078,928$).

En ciencia de datos, los while se usan menos que los for para iterar sobre colecciones (ya que generalmente sabemos el tamaño del conjunto de datos). Sin embargo, pueden ser útiles en casos en que se lee un stream de datos hasta que se agote, o en bucles de procesamiento que deben esperar cierta condición de convergencia, etc.

Imagen 2: ¿Cuándo usar while/for?



Fuente: imagen creada con IA

Ejemplo integrador de control: Imaginemos que tenemos una lista de mediciones de calidad del aire (índice ICA) diarias en Santiago, y queremos imprimir una advertencia en cada valor que exceda cierto nivel (por ejemplo, 150, considerado nocivo):

```
ica_diario = [120, 80, 200, 50, 300] # índices de un periodo (ficticios)
for valor in ica_diario:
    if valor > 150:
        print(valor, "→ ALERTA: Calidad del aire nociva")
    else:
        print(valor, "→ OK")
```

La salida sería:

120 → OK
80 → OK
200 → ALERTA: Calidad del aire nociva
50 → OK
300 → ALERTA: Calidad del aire nociva

Aquí combinamos for para iterar la lista con if dentro para decidir el mensaje según el valor. Este es un ejemplo práctico de uso conjunto de estructuras de control para automatizar un proceso de análisis (evaluar cada medición y marcar las que son peligrosas), logrando lo que indica el resultado de aprendizaje 1.1.2 (uso de estructuras de control para automatizar procesos garantizando coherencia; en este caso, la coherencia sería que a todo valor >150 se le aplica la misma regla de alerta).

3. Definición y uso de funciones

A medida que los programas crecen en tamaño y complejidad, es fundamental organizarlos de forma modular. Las funciones en Python nos permiten agrupar un conjunto de instrucciones bajo un nombre, de modo que ese "bloque" de código puede reutilizarse fácilmente varias veces, incluso con datos diferentes. En cursos previos quizás has visto la idea de subrutinas o funciones matemáticas; en programación, una función recibe entradas (parámetros), realiza acciones (algoritmo) y opcionalmente devuelve un resultado. Esto contribuye a escribir código más limpio, evitar repeticiones y facilitar la reutilización y mantenimiento.

3.1 ¿Qué es una función y por qué usarla?

Una función es como una "caja" que recibe ciertos datos de entrada, realiza operaciones con ellos y puede entregar un resultado de salida. Definimos una función una vez y luego podemos "invocarla" o llamarla múltiples veces en distintos contextos, en lugar de reescribir el mismo código una y otra vez. Esto es esencial para modularizar tareas. Por ejemplo, si analizamos datos de distintas fuentes, podríamos escribir una función `limpiar_datos(datos)` que aplique ciertas transformaciones y limpieza; luego esa misma función puede usarse con varios datasets, garantizando coherencia en el proceso de limpieza (logrando la consistencia de la que habla el resultado 1.1.2).

Además, las funciones hacen más legible el código: en lugar de ver 20 líneas que realizan una tarea específica, vemos una llamada del tipo `procesar_archivo("datos.csv")`, lo que nos indica claramente qué se está haciendo, delegando el detalle a la definición de la función.

3.2 Definiendo funciones en Python

En Python se define una función usando la palabra clave `def`, seguida del nombre de la función, parámetros entre paréntesis y dos puntos. El cuerpo de la función va indentado debajo. Para devolver un valor se usa la sentencia `return`. Si no se especifica `return`, la función devuelve implícitamente `None` (un tipo especial que indica "nada"/"vacío").

Ejemplo de una función simple que no toma parámetros y no devuelve nada (solo imprime algo):

```
def saludar():  
    print("Hola, bienvenido al sistema")
```

Aquí definimos la función `saludar`. Para ejecutarla (llamarla), simplemente escribimos `saludar()`. Cada vez que la llamemos, imprimirá el mensaje.

Veamos funciones con parámetros y retorno, que son más útiles.

Ejemplo 1: Función para calcular el promedio (media) de una lista de números:

```
def promedio(lista_numeros):  
    """Calcula y retorna el promedio de una lista de números."""  
    total = 0  
    for n in lista_numeros:  
        total += n  
    cantidad = len(lista_numeros)  
    if cantidad == 0:  
        return 0 # evitar división por cero, definimos promedio de lista vacía como 0  
    return total / cantidad
```

Explicación: La función promedio toma un parámetro lista_numeros. Hemos incluido un comentario entre triple comillas "" al inicio de la función, llamado docstring, que describe brevemente la función (esto es buena práctica, aunque opcional, para documentar). Dentro de la función inicializamos total, iteramos con un for sumando cada número, luego dividimos por la cantidad de elementos y retornamos el resultado. Si la lista estuviese vacía (len(lista_numeros) == 0), retornamos 0 para evitar error de división.

Ahora podemos usar esta función en nuestro programa:

```
datos = [5, 10, 15]
prom = promedio(datos)
print("El promedio es:", prom) # Debería imprimir "El promedio es: 10.0"
```

La variable prom recibirá el valor devuelto por promedio(datos), que en este caso sería 10.0.

Ejemplo 2: Función para limpiar una lista de datos numéricos, reemplazando valores negativos con 0 (imaginemos que en ciertos datos, los negativos son entradas erróneas):

```
def limpiar_negativos(lista_valores):
    """
    Recorre una lista de números y reemplaza los valores negativos por 0.
    Retorna la lista limpiada.
    """
    for i, valor in enumerate(lista_valores):
        if valor < 0:
            lista_valores[i] = 0
    return lista_valores
```

Aquí usamos enumerate para tener tanto el índice i como el valor en la lista durante la iteración. Cuando encontramos un valor negativo (valor < 0), lo reemplazamos asignando 0 en esa posición. Finalmente retornamos la lista modificada. Un punto a destacar: las listas se pasan por referencia (es decir, si modificamos lista_valores dentro, la lista original fuera de la función también se verá modificada). En este caso retornamos la lista por comodidad, pero estrictamente no sería necesario retornarla si entendemos que la lista original ya quedó modificada. Sin embargo, retornar permite encadenar o asignar el resultado explícitamente.

Veamos cómo usar esta función:

```
mediciones = [12, -5, 7, -3, 15] # supongamos mediciones donde -5 y -3 son errores
limpiar_negativos(mediciones)
print("Mediciones limpias:", mediciones)
```

La salida sería: Mediciones limpias: [12, 0, 7, 0, 15]. Los valores negativos se sustituyeron por 0. Este es un ejemplo sencillo de automatización de limpieza de datos utilizando una función (relacionado con el aprendizaje 1.1.2: estamos garantizando consistencia al tratar todos los negativos bajo la misma regla).

Ejemplo: Supongamos que trabajamos con temperaturas en grados Celsius y queremos convertir a Fahrenheit repetidamente. Podemos definir una función para convertir una temperatura:

```
def celsius_a_fahrenheit(celsius):  
    """Convierte temperatura de Celsius a Fahrenheit."""  
    return celsius * 9/5 + 32  
  
# Usando la función con temperaturas de ejemplo  
temps_C = [0, 20, 37] # 0°C, 20°C, 37°C (aprox temp corporal)  
for t in temps_C:  
    print(t, "°C ->", celsius_a_fahrenheit(t), "°F")  
Salida:  
0 °C -> 32.0 °F  
20 °C -> 68.0 °F  
37 °C -> 98.6 °F
```

Aquí definimos `celsius_a_fahrenheit` y luego la utilizamos dentro de un loop para convertir tres valores de prueba. Esto ilustra la reutilización: sin la función, tendríamos que escribir la fórmula 3 veces; con la función, simplemente la llamamos con diferentes valores.

3.3 Alcance de las variables y parámetros

Es útil mencionar brevemente que las variables creadas *dentro* de una función (como `total` en la función `promedio`) son locales a esa función: no existen fuera de ella. De igual modo, los parámetros (como `lista_numeros`) son locales. Si en el programa principal tenemos una variable `total`, no interferirá con la `total` dentro de la función. Este aislamiento ayuda a prevenir conflictos de nombres.

Las funciones pueden llamar a otras funciones, incluso a sí mismas (recursividad), pero eso último es más avanzado y no lo necesitaremos en este nivel inicial.

Sugerencia de imagen: Un diagrama de caja negra de una función, mostrando cómo entran los parámetros y sale el resultado, o un diagrama de stack (pila de llamadas) donde se vea el ámbito local de una función. Por ejemplo, para la función `promedio`, ilustrar que se le pasa una lista, realiza cálculos internamente, y devuelve un número.

4. Manejo básico de archivos y entrada/salida de datos

Hasta ahora hemos trabajado con datos "en memoria" definidos directamente en el código (listas, variables con valores, etc.). En ciencia de datos, la información suele provenir de fuentes externas: archivos en disco (CSV, Excel, JSON), bases de datos, APIs, etc. En esta sección nos enfocaremos en cómo leer y escribir archivos desde Python, particularmente archivos de texto y CSV, ya que son formatos comunes para datasets. Esto nos permitirá cargar datos reales en nuestros programas (cumpliendo con el resultado 1.1.1 de ejecutar programas que carguen datos) y también guardar resultados procesados en archivos para su posterior uso o reporte.

Python proporciona funciones integradas para entrada/salida (I/O) de archivos. La función principal para manejar archivos es `open()`. Veamos cómo usarla para lectura y escritura.

4.1 Leyendo datos desde archivos

La sintaxis básica para abrir un archivo es:

```
archivo = open("ruta/al/archivo.ext", modo)
```

Donde *modo* suele ser "r" para leer (*read*), "w" para escribir (*write*), "a" para agregar (*append*) al final, entre otros. Es buena práctica usar la estructura `with` al trabajar con archivos, ya que se asegura de cerrar el archivo automáticamente luego de usarlo:

```
with open("datos.txt", "r") as arch:
    # dentro de este bloque podemos leer del archivo 'arch'
    # al salir del with, el archivo se cierra automáticamente
```

Una vez abierto un archivo en modo lectura, tenemos varias formas de leer su contenido:

- `arch.read()` devuelve todo el contenido del archivo como una sola cadena (útil para archivos pequeños, pero no ideal con archivos muy grandes).
- `arch.readline()` lee una línea completa (hasta encontrar un salto de línea `\n`) y la devuelve como cadena.
- Iterar sobre el archivo directamente: `for linea in arch:` irá entregando cada línea del archivo en cada iteración, lo cual es eficiente y cómodo.

Supongamos que tenemos un archivo llamado "regiones.csv" que contiene datos tabulares simples (por ejemplo, el mismo dataset de regiones que usamos antes con columnas de región, población, superficie). Vamos a leerlo línea por línea e imprimir su contenido:

```
with open("regiones.csv", "r") as arch:
    encabezado = arch.readline() # leer la primera línea (cabecera) y descartarla
    for linea in arch:
        linea = linea.strip() # eliminar el salto de línea final y espacios extra
        print(linea)
```

Si el archivo `regiones.csv` tuviese, por ejemplo, el siguiente contenido:

```
region,poblacion,superficie
Metropolitana,7100000,15403
Valparaiso,1800000,16396
Biobio,1550000,23890
Araucania,1000000,31842
Atacama,300000,74863
Aysen,110000,108494
```

La salida del código anterior sería cada línea de datos impresa:

```
Metropolitana,7100000,15403
Valparaiso,1800000,16396
Biobio,1550000,23890
Araucania,1000000,31842
Atacama,300000,74863
Aysen,110000,108494
```

(cada línea del archivo original se muestra, excepto la cabecera que saltamos intencionalmente con `arch.readline()`).

En este ejemplo, leímos el archivo CSV como texto plano. Cada línea es una cadena con valores separados por coma. Podríamos aprovechar esto para parsear los datos: por ejemplo, podríamos usar `linea.split(',')` para convertir cada línea en una lista de valores separados:

```
with open("regiones.csv", "r") as arch:
    arch.readline() # saltar cabecera
    for linea in arch:
        campos = linea.strip().split(',')
        print(campos)
```

La salida entonces serían listas:

```
['Metropolitana', '7100000', '15403']
['Valparaiso', '1800000', '16396']
['Biobio', '1550000', '23890']
```

Vemos que cada lista contiene los datos de la región, pero ten en cuenta que los números estarán como cadenas ('7100000' y '15403' en lugar de enteros). Podríamos convertirlos con `int()` si quisiéramos usarlos numéricamente.

Esta es una forma manual de leer un CSV. Python también tiene el módulo estándar `csv` que facilita manejar archivos CSV considerando comillas, separadores, etc. Por ejemplo, usando `csv.reader`:

```
import csv
with open("regiones.csv", "r") as arch:
    reader = csv.reader(arch)
    for fila in reader:
        print(fila)
```

La salida sería similar (incluyendo esta vez la cabecera como primera fila). Para simplicidad en este nivel, la lectura manual con `split(',')` es comprensible, pero es bueno saber que el módulo `csv` existe para casos más complejos (por ejemplo, campos de texto que contienen comas y van entrecomillados, etc.).

Ejemplo: Podríamos tener un archivo `estaciones.txt` con nombres de estaciones de metro de Santiago en cada línea. Con el método mostrado, podríamos leerlo fácilmente:

```
with open("estaciones.txt", "r") as f:
    for linea in f:
        estacion = linea.strip()
        print("Estación:", estacion)
```

Esto imprimiría cada estación precedida del texto "Estación:". Es un ejemplo simple de lectura secuencial de un archivo de texto línea por línea.

4.2 Escribiendo datos a archivos

De forma análoga a la lectura, podemos escribir en archivos abriéndolos en modo escritura ("w"). Al abrir en modo "w", si el archivo ya existe se sobrescribe (se borra su contenido anterior); si no existe, se crea uno nuevo vacío. Existe también el modo "a" (append) para agregar contenido al final de un archivo existente sin borrar lo anterior.

La función principal es `f.write(cadena)` que escribe la cadena dada en el archivo (tal cual, sin añadir nada extra). Hay que asegurarse de incluir saltos de línea `\n` manualmente si queremos escribir en líneas separadas.

Ejemplo básico:

```
resultado = 42
with open("salida.txt", "w") as salida:
    salida.write("El resultado del cálculo es: " + str(resultado) + "\n")
    salida.write("Proceso finalizado exitosamente.\n")
```

Aquí creamos/abrimos `salida.txt` en modo escritura. Luego escribimos dos líneas. La primera combina texto fijo con el número resultado convertido a cadena (usando `str`), y añadimos `\n` al final para el salto de línea. La segunda línea solo escribe un mensaje fijo con su propio `\n`. Después de ejecutar este código, el archivo `salida.txt` contendrá exactamente:

El resultado del cálculo es: 42

Proceso finalizado exitosamente.

Otra forma útil es preparar todo el contenido en una sola cadena multilínea o en una lista y usar `writelines`. Por ejemplo:

```
lineas = [
    "Línea 1: Hola\n",
    "Línea 2: Este es un archivo de salida\n",
    "Línea 3: Adiós\n"
]
with open("mensaje.txt", "w") as f:
    f.writelines(lineas)
```

En este caso, escribimos tres líneas de golpe. Aseguramos que cada línea en la lista líneas ya termine con `\n` para separar líneas.

Ejemplo: Si procesamos datos de ventas de una empresa y calculamos que la venta total del mes fue X pesos, podríamos generar un pequeño reporte de texto:

```
ventas_total = 12500000 # por ejemplo, $12.500.000 CLP
with open("reporte.txt", "w") as rep:
    rep.write("Resumen de Ventas Mensuales\n")
    rep.write("=====\n")
    rep.write(f"Ventas totales: {ventas_total} pesos\n")
    rep.write("Promedio diario: " + str(ventas_total/30) + " pesos\n")
```

Observamos que aquí combinamos distintas formas: texto estático, el operador f string (f-string) para incrustar la variable `ventas_total` dentro de una cadena (otra forma conveniente de formatear cadenas en Python 3+), y concatenación manual con `str()`. Cualquiera es válida; las f-strings suelen ser muy prácticas para generar salidas formateadas.

Luego de ejecutar ese código, `reporte.txt` tendría algo parecido a:

```
Resumen de Ventas Mensuales
=====
Ventas totales: 12500000 pesos
Promedio diario: 416666.6666666667 pesos
```

(El promedio diario se ve con muchos decimales; podríamos formatearlo mejor, pero eso es detalle de presentación.)

4.3 Ejemplo: lectura, procesamiento simple y escritura

Para consolidar, hagamos un pequeño ejercicio de flujo completo: leer un archivo, hacer un cálculo simple y guardar el resultado en otro archivo. Esto ilustrará la secuencia lectura → procesamiento → salida.

Supongamos que tenemos un archivo `numeros.txt` con un número entero en cada línea (pueden ser datos de producción, cantidades, etc.). Queremos sumar esos números y guardar el total en `suma.txt`.

Lectura y suma:

```
total = 0
with open("numeros.txt", "r") as f:
    for linea in f:
        numero = int(linea.strip()) # convertimos cada línea a int
        total += numero
print("Suma calculada:", total)
```

Aquí leemos cada línea, la convertimos a entero (asumiendo que cada línea es un número válido) y acumulamos. Imprimimos la suma por pantalla para verificar.

Escritura del resultado:

```
with open("suma.txt", "w") as f:
    f.write(f"La suma de los números es: {total}\n")
Esto crea/escibe el resultado en suma.txt.
```

Importante: Antes de ejecutar este código en la vida real, debemos asegurar que `numeros.txt` existe y tiene el formato esperado. Este ejemplo es una simplificación: no maneja posibles errores (como líneas vacías o datos no numéricos). En un contexto de garantizar coherencia y consistencia de datos (LO 1.1.2), uno querría quizás agregar chequeos dentro del `for`: por ejemplo, ignorar líneas que no sean numéricas, o acumular en un contador de errores si algo no se puede convertir a `int`. Pero eso escapa al alcance básico; por ahora asumimos datos limpios.

Imagen 3: flujo de leer-procesar-escribir



Fuente: imagen creada con IA

5. Introducción a librerías especializadas para manipulación de datos: NumPy y pandas

Python cuenta con un ecosistema riquísimo de librerías (módulos externos) orientadas a la ciencia de datos. Dos de las más destacadas son NumPy y pandas. NumPy proporciona estructuras y funciones para computación numérica eficiente (principalmente arreglos de números n-dimensionales, similares a vectores y matrices matemáticas). Por su parte, pandas está construida sobre NumPy y ofrece estructuras de datos de alto nivel (principalmente el DataFrame, que es una tabla de datos similar a una hoja de cálculo o tabla SQL) junto con múltiples operaciones para facilitar la manipulación y análisis de datos tabulares.

Para usar una librería externa, primero se debe instalar (muchas ya vienen con distribuciones científicas como Anaconda) y luego importarla en el script. El import se hace con la sintaxis:

```
import nombre_libreria as alias
```

Es costumbre importar NumPy como np y pandas como pd para abreviar.

A continuación, damos un panorama básico de cada una:

5.1 NumPy: arreglos numéricos eficientes

NumPy (Numerical Python) introduce el tipo de dato ndarray (arreglo n-dimensional), usualmente referido simplemente como *array*. Un array de NumPy es similar a una lista en cuanto a que contiene una colección de elementos, pero *todos* los elementos deben ser del mismo tipo (generalmente números) y está implementado de manera contigua en memoria, lo que lo hace mucho más eficiente para operaciones matemáticas masivas.

Ventajas principales de NumPy:

- Operaciones vectorizadas: se pueden realizar operaciones aritméticas sobre los arrays directamente, aplicándose elemento a elemento, mucho más rápido que con bucles de Python.
- Funciones matemáticas optimizadas (suma, media, seno, etc. aplicados a arrays completos).
- Soporte de arrays multi-dimensionales (matrices).
- Muchas funcionalidades algebraicas (producto de matrices, transformadas, etc., aunque eso escapa a nuestro enfoque básico).

Veamos algunos ejemplos:

```
Primero importemos NumPy:
import numpy as np
Ahora, crear un array:
lista = [1, 2, 3, 4]
arr = np.array(lista)
print("Array de NumPy:", arr)
Salida: Array de NumPy: [1 2 3 4]. Parece similar a imprimir una lista, pero arr es de
tipo numpy.ndarray. Podemos verificar:
print(type(arr)) # -> <class 'numpy.ndarray'>
```

Hagamos una operación vectorizada:

```
# Multiplicar cada elemento por 2
print("Lista * 2:", lista * 2)
print("Array * 2:", arr * 2)
Esto producirá:
Lista * 2: [1, 2, 3, 4, 1, 2, 3, 4]
Array * 2: [2 4 6 8]
```

¿Por qué la diferencia? En Python, la operación `lista * 2` repite la lista (concatenándola consigo misma), porque para listas el operador `*` está definido como repetición. En cambio, en un `numpy.array`, el `*` realiza multiplicación elemento a elemento (también llamada multiplicación escalar por 2 en este caso). Así, NumPy interpreta `arr * 2` como `[12, 22, 32, 42] -> [2, 4, 6, 8]`. Esto demuestra la comodidad de NumPy para cálculos: no necesitamos recorrer manualmente para multiplicar; la operación se aplica a todos de una vez.

Más operaciones con arrays:

```
arr2 = np.array([10, 20, 30, 40])
# Suma de arrays (elemento a elemento)
print("Suma:", arr + arr2)    # -> [11 22 33 44]
```

Comparaciones vectorizadas

```
print("Comparación:", arr2 > 25) # -> [False False  True  True] (un array booleano resultante)
```

Funciones matemáticas

```
print("Promedio de arr2:", np.mean(arr2)) # -> 25.0
print("Seno de arr:", np.sin(arr))        # aplica seno a cada elemento de arr
```

NumPy es especialmente útil para arreglos grandes (cientos o miles de elementos), donde las operaciones vectorizadas son mucho más rápidas que hacer bucles en Python puro.

Ejemplo: Supongamos que tenemos los ingresos mensuales (en millones de pesos) de una startup chilena durante 5 años, en una lista de 60 valores (12 meses * 5 años). Queremos calcular rápidamente algunas estadísticas. Con NumPy:

```
import numpy as np
ingresos = np.array([3.2, 4.1, 5.0, ... ]) # (imaginemos 60 valores aquí)
print("Ingreso promedio mensual:", np.mean(ingresos))
print("Desviación estándar:", np.std(ingresos))
# Supongamos además que queremos proyectar un crecimiento del 10% en todos los meses:
proyeccion = ingresos * 1.1
```

En una línea calculamos la proyección multiplicando todo el array por 1.1, en lugar de iterar mes por mes.

Aunque podríamos profundizar mucho más en NumPy, para los fines de esta unidad nos interesa más su uso como base para pandas. Pensemos en NumPy arrays como la versión optimizada de las listas para números, y tengamos en mente que muchas operaciones con pandas (filtros, cálculos por columna) ocurren utilizando esta eficiencia por debajo.

5.2 pandas: DataFrames para datos tabulares

Pandas es una librería diseñada para el análisis y manipulación de datos tabulares (estructurados en filas y columnas). Su estructura fundamental es el DataFrame, que podemos imaginar como una tabla (similar a una hoja de cálculo de Excel o a una tabla en SQL), donde cada columna puede ser de un tipo diferente (números, texto, fechas, etc.) pero todos los datos de una columna comparten tipo, y cada fila representa una instancia u observación (por ejemplo, un individuo, un día de registro, etc.).

Otra estructura importante en pandas es la Serie (Series), que esencialmente es una columna de datos con un índice (es como un array de NumPy pero con etiqueta para cada elemento). De hecho, cada columna de un DataFrame es una Series.

Primero, importemos pandas:

```
import pandas as pd
```

Creación de un DataFrame

Podemos crear un DataFrame manualmente a partir de un diccionario de listas, donde las claves serán los nombres de columnas y los valores las listas de datos:

```
datos_ejemplo = {  
    "Region": ["Metropolitana", "Valparaíso", "Biobío"],  
    "Poblacion": [7100000, 1800000, 1550000],  
    "Superficie": [15403, 16396, 23890]  
}  
df = pd.DataFrame(datos_ejemplo)  
print(df)
```

Salida:

	Region	Poblacion	Superficie
0	Metropolitana	7100000	15403
1	Valparaíso	1800000	16396
2	Biobío	1550000	23890

Aquí pandas ha construido una tabla con tres filas (0,1,2 como índices por defecto) y tres columnas. Observemos:

- La primera columna mostrada sin nombre es el índice de las filas (0,1,2). Si no lo especificamos, pandas asigna índices numéricos empezando en 0.
- Las columnas tienen nombres "Region", "Poblacion", "Superficie".
- Pandas al imprimir alinea las columnas y muestra cada fila en una línea.

Este DataFrame df contiene los datos de las regiones Metropolitana, Valparaíso y Biobío. Podríamos haber especificado un índice personalizado (por ejemplo, usando la columna Region como índice), pero por ahora mantengámoslo simple.

Acceder a columnas:

```
print(df["Region"]) # imprime la columna 'Region' (esto es una Series)
print(df.Poblacion) # otra sintaxis: df.NombreCol (válido si el nombre no tiene
                    espacios, etc.)
```

La primera línea imprimiría algo como:

```
0  Metropolitana
1   Valparaíso
2     Biobío
```

Name: Region, dtype: object

Esto es la representación de una Series: muestra índice 0,1,2 y los valores de cada, con dtype (tipo) object que en pandas suele indicar cadenas.

Notemos que `df["Region"]` es similar a un diccionario donde clave es el nombre de columna. La sintaxis `df.Poblacion` es azúcar sintáctica para lo mismo (como un atributo), pero solo funciona con nombres de columna que sean strings válidos como identificador y sin espacios.

¿Por qué pandas es útil?

- Lectura de archivos fácil: pandas puede leer CSV, Excel, JSON, SQL, etc., y convertirlos directamente en DataFrames en una línea de código. Por ejemplo `pd.read_csv("datos.csv")` lee un CSV y devuelve un DataFrame con los datos.
- Operaciones rápidas en columnas: filtrar filas según condiciones, calcular columnas derivadas, agrupar datos, etc., todo con sintaxis concisa.
- Integración con bibliotecas de visualización: es sencillo generar gráficos a partir de DataFrames o Series.
- Manejo de valores faltantes (NaN), merges (combinar datasets), reordenamientos, muchas funciones estadísticas, etc.

En la siguiente sección exploraremos operaciones fundamentales de pandas en detalle (carga, exploración, filtrado, etc.). Por ahora, veamos un pequeño ejemplo para motivar:

Ejemplo: Supongamos que tenemos un dataset con información de automóviles vendidos en Chile, con columnas como "Marca", "Modelo", "Precio", "Año". Con pandas, podríamos cargarlo y realizar consultas simples como "filtrar todos los autos de marca Toyota de año 2020 en adelante". Usando sintaxis pandas eso podría ser una línea:

```
autos = pd.read_csv("autos.csv") # cargar datos desde un CSV
toyota_2020 = autos[(autos["Marca"] == "Toyota") & (autos["Año"] >= 2020)]
print(toyota_2020.head())
```

En esa línea, `autos["Marca"] == "Toyota"` produce una serie booleana con True/False según la marca, similar para `Año >= 2020`. Con el `&` (and bit a bit) combinamos ambas condiciones. El resultado `toyota_2020` sería un DataFrame filtrado con solo las filas que cumplen ambas condiciones. `head()` luego mostraría las primeras filas de ese resultado (tal vez 5 filas por defecto) para inspección.

Como se aprecia, esto evita tener que iterar manualmente por cada fila. Internamente pandas y NumPy hacen esas comparaciones de forma vectorizada, por lo que incluso con miles de registros es muy eficiente.

Imagen 4: Ejemplo de un dataframe

ÍNDICE	MARCA	MODELO	AÑO	PRECIO	DISPONIBLE
0	Honda	Civic	2019	\$15,000	Sí
1	Toyota	Corolla	2021	\$22,000	Sí
2	Ford	Focus	2018	\$12,000	No
3	Toyota	RAV4	2022	\$30,000	Sí
4	Nissan	Sentra	2020	\$18,000	Sí

Fuente: imagen creada con IA

6. Operaciones fundamentales de manipulación de datos con pandas

En esta sección pondremos en práctica la manipulación de datos usando pandas, siguiendo el flujo típico de trabajo con datos tabulares:

1. Carga de datos en un DataFrame.
2. Exploración inicial: vistazo a las primeras filas, conocer tipos de datos, tamaño, etc.
3. Selección y filtrado: extraer subconjuntos de datos, sea por columnas o por filas que cumplen condiciones.
4. Ordenamiento: reordenar las filas según valores de una o más columnas.
5. Agregación simple: calcular estadísticas resumidas, como sumas, promedios, conteos, etc., posiblemente para todo el DataFrame o por grupos simples.

Usaremos de ejemplo el dataset de regiones chilenas que ya mencionamos, para ser consistentes. Imaginemos que guardamos ese dataset en un archivo CSV llamado "regiones.csv" con las columnas region, poblacion, superficie (como el mostrado en la sección de archivos). Ahora realizaremos las operaciones con pandas.

6.1 Carga de datasets en DataFrames

La función más común para leer archivos CSV es `pd.read_csv()`. Si el archivo tiene encabezados en la primera fila (nombres de columna), pandas los utiliza automáticamente. Debemos asegurarnos de indicar la ruta correcta del archivo, o que esté en el directorio de trabajo.

```
import pandas as pd
df = pd.read_csv("regiones.csv")
print("Datos cargados:")
print(df.head())
```

Explicación:

- Importamos pandas como `pd`.
- Usamos `pd.read_csv("regiones.csv")` para leer el archivo. Esto devuelve un DataFrame que almacenamos en la variable `df`.
- Usamos `df.head()` para mostrar las primeras 5 filas del DataFrame (por defecto `head()` muestra 5, pero se puede pasar un número dentro de los paréntesis para mostrar ese número de filas).

Si `regiones.csv` contenía las 6 regiones de ejemplo, `head()` mostrará algo así:

	region	población	superficie
0	Metropolitana	7100000	15403
1	Valparaíso	1800000	16396
2	Biobío	1550000	23890
3	Araucanía	1000000	31842
4	Atacama	300000	74863

Vemos las columnas `region`, `poblacion`, `superficie`. La salida de `head()` muestra 5 filas (índices 0 a 4). Como el DataFrame tenía 6 filas totales, la última (índice 5, Aysén) no aparece aquí, pero sabemos que está cargada.

Nota: Al imprimir un DataFrame entero, si es grande pandas suele resumirlo mostrando las primeras y últimas filas y usando ... en medio. `head()` es útil para ver solo el inicio, y `df.tail()` muestra las últimas filas.

Pandas infiere tipos de datos de las columnas: en este caso, seguramente "region" será tipo objeto (string), "poblacion" y "superficie" serán numéricos (int64).

6.2 Exploración inicial (inspección de datos)

Después de cargar un dataset, es buena práctica explorar su contenido y propiedades:

- Tamaño (filas, columnas): usando `df.shape` obtendremos una tupla (`n_filas`, `n_columnas`).
- Tipos de datos por columna: `df.dtypes` lista los tipos.
- Información general: `df.info()` da un resumen con número de filas, columnas, tipos, y cuentas de valores no nulos.
- Estadísticas descriptivas rápidas: `df.describe()` calcula estadísticas básicas (count, mean, std, min, max, percentiles) para columnas numéricas.

Aplicando a `df`:

```
print("Dimensiones (filas, columnas):", df.shape)
print("\nTipos de datos:")
print(df.dtypes)
print("\nInformación completa:")
print(df.info())
print("\nEstadísticas descriptivas:")
print(df.describe())
```

Posible salida:

```
Dimensiones (filas, columnas): (6, 3)
Tipos de datos:
region      object
poblacion   int64
superficie  int64
dtype: object

Información completa:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   region      6 non-null     object
1   poblacion   6 non-null     int64
2   superficie  6 non-null     int64
dtypes: int64(2), object(1)
memory usage: 272.0+ bytes
None
```

Estadísticas descriptivas:

```
      poblacion  superficie
count 6.000000e+00    6.000000
mean  1.976667e+06  46399.666667
std    2.602596e+06  40964.755992
min    1.100000e+05  15403.000000
25%    5.375000e+05  20281.250000
50%    1.275000e+06  27866.000000
75%    1.790000e+06  58198.250000
max    7.100000e+06  108494.000000
```

Analicemos:

- `shape` indica 6 filas, 3 columnas.
- `dtypes` confirma "region" es object (tipo string), las otras dos `int64`.
- `info()` muestra no hay valores nulos (Non-Null Count = 6 en cada columna, coincidiendo con 6 filas, indica datos completos). También muestra la memoria usada (irrelevante para tan pocos datos, pero útil en data grande).
- `describe()` proporciona stats para columnas numéricas: `count=6` (hay 6 valores), media `mean`, desviación `std`, `min`, `max`, y los cuartiles 25%, 50% (mediana), 75%. Por ejemplo, la media de población $\sim 1.97 \times 10^6$ (1.97 millones), el mínimo 110000 (Aysén), máximo 7100000 (Metropolitana). En superficie, `min` 15403 km² (RM), `max` 108494 km² (Aysén), etc. Estas estadísticas permiten entender la distribución de los datos numéricos.

Con una exploración rápida ya identificamos posibles outliers o rangos: por ejemplo, la superficie media ~ 46399 km² sugiere hay regiones mucho más grandes que otras (se confirma con `max` 108494 vs `min` 15403). De hecho, Aysén es enorme en área pero muy pequeña en población, vs Metropolitana pequeña área pero gran población. Esto sugiere variabilidad que podríamos analizar calculando densidad, etc.

6.3 Selección y filtrado de datos

Selección de columnas: Ya vimos que podemos obtener una columna usando `df["nombre_col"]`. Si deseamos seleccionar múltiples columnas, podemos pasar una lista de nombres:

```
cols_interes = ["region", "poblacion"]
df_sub = df[cols_interes]
print(df_sub.head())
```

Esto crearía un nuevo DataFrame `df_sub` solo con las columnas "region" y "poblacion". En nuestro ejemplo:

```
   region poblacion
0  Metropolitana  7100000
1   Valparaiso   1800000
2     Biobio    1550000
3  Araucania    1000000
4    Atacama     300000
(ha omitido "superficie").
```

Selección de filas por posición: Podemos usar índices numéricos para filas. Con `df.iloc[filas_idx]` obtenemos la fila por índice (similar a listas). Por ejemplo `df.iloc[0]` sería la primera fila (Metropolitana), `df.iloc[-1]` la última (Aysén). También `df.iloc[2:5]` daría un subconjunto (fila 2 a 4). Sin embargo, raramente necesitamos usar índices numéricos directos en ciencia de datos, más común es filtrar por contenido.

Filtrado de filas por condición: Una de las operaciones más poderosas: podemos crear una máscara booleana usando condiciones sobre las columnas, y usarla entre corchetes para filtrar.

Por ejemplo, obtengamos las regiones con población sobre 1 millón:

```
filtro = df["poblacion"] > 1000000
print("Máscara booleana:\n", filtro)
grandes = df[filtro]
print("\nRegiones con poblacion > 1.000.000:")
print(grandes)
```

Primero calculamos filtro, que es una Series booleana True/False para cada fila:

```
0   True  (Metropolitana 7100000 > 1000000? True)
1   True  (Valparaíso 1800000 > 1000000? True)
2   True  (Biobío 1550000 > 1000000? True)
3  False  (Araucanía 1000000 > 1000000? False, es igual, no mayor estricto)
4  False  (Atacama 300000 > 1000000? False)
5  False  (Aysén 110000 > 1000000? False)
```

Name: poblacion, dtype: bool

Luego df[filtro] devuelve un DataFrame con solo las filas donde filtro es True:

	region	poblacion	superficie
0	Metropolitana	7100000	15403
1	Valparaiso	1800000	16396
2	Biobio	1550000	23890

Se listan las tres regiones con más de 1 millón de hab.

Notar que Araucanía tiene exactamente 1,000,000 y quedó fuera porque usamos > 1000000. Si quisiéramos incluirla usaríamos >=. Podemos combinar condiciones lógicas con operadores bit a bit & (and), | (or), ~ (not) sobre las Series booleanas. Por ejemplo, regiones entre 500k y 2M de población:

```
filtro2 = (df["poblacion"] >= 500000) & (df["poblacion"] <= 2000000)
medianas = df[filtro2]
print("Regiones con población de 0.5 a 2 millones:")
print(medianas)
```

La máscara combina dos condiciones con &. Resultado:

	region	poblacion	superficie
1	Valparaiso	1800000	16396
2	Biobio	1550000	23890
3	Araucania	1000000	31842

(Metropolitana fuera por >2M, Atacama y Aysén fuera por <500k).

Selección por índice/etiqueta: En nuestro DataFrame el índice es numérico 0-5. Si la columna "region" fuera índice, podríamos usar df.loc["Atacama"] para acceder a la fila de Atacama. df.loc permite seleccionar por etiqueta de índice y también combinar con condiciones. Dado que no establecimos un índice particular, df.loc con fila funciona similar a df.iloc (0-based). Sin profundizar, mencionamos que loc se usa cuando queremos seleccionar por nombre de índice o por nombre de columnas en simultáneo.

Modificar valores al filtrar: Podemos asignar un nuevo valor a todas las filas que cumplan cierta condición. Por ejemplo, podríamos corregir un dato erróneo asignando:

```
df.loc[df["region"] == "Valparaiso", "region"] = "Valparaíso"
```

(si notamos que faltaba tilde, esto reemplaza "Valparaiso" por "Valparaíso" en la columna region donde coincide esa fila). Este tipo de operación muestra cómo pandas facilita limpieza de datos específicos.

6.4 Ordenamiento de datos

Ordenar (o sortear) datos por valores de una o varias columnas es útil para ver por ejemplo el top de algo (e.g., regiones más pobladas, etc.). En pandas usamos `df.sort_values(by="columna")`. Por defecto es ascendente, podemos agregar `ascending=False` para descendente.

Ordenemos nuestro DataFrame por población de mayor a menor:

```
df_sorted = df.sort_values(by="poblacion", ascending=False)
print("Regiones ordenadas por población (desc).")
print(df_sorted)
```

Salida:

	region	poblacion	superficie
0	Metropolitana	7100000	15403
1	Valparaiso	1800000	16396
2	Biobio	1550000	23890
3	Araucania	1000000	31842
4	Atacama	300000	74863
5	Aysen	110000	108494

Efectivamente quedó de mayor a menor población (Metropolitana primero, Aysén último). Observemos que el índice original se mantiene con los números originales asociados a cada fila; es decir, no reindexa automáticamente. En la impresión, la primera fila sigue marcando índice 0 (Metropolitana), luego 1,2,... en orden de población. Esto es algo a tener en cuenta: podemos usar `df_sorted = df.sort_values(..., ignore_index=True)` si queremos que reasigne índices 0..n-1 tras ordenar, pero en general no es necesario a menos que se requiera.

También se puede ordenar por múltiples columnas, pasando una lista a `by=[col1, col2]`. Por ejemplo, podríamos ordenar por región alfabéticamente: `df.sort_values(by="region")` (Valparaíso vendría antes de Valparaiso? cuidado acentos: probablemente lo ordena lexicográficamente con ASCII, la tilde podría afectar el orden; pandas quizás trate object sorting dependiendo de locale, pero no nos adentraremos mucho). O `sort multi-col`: si tuviéramos una columna de categoría y otra numérica, podríamos ordenar por categoría y dentro de cada categoría por la numérica.

6.5 Agregación simple de valores

Con *agregación* nos referimos a calcular algún valor resumen a partir de una serie de valores. Pandas DataFrame y Series ofrecen métodos como:

- `sum()`: suma de valores (por columnas si es DataFrame).
- `mean()`: promedio.
- `median()`, `min()`, `max()`, `std()` etc.
- `count()`: número de valores no nulos.
- `value_counts()`: (en Series) cuenta las frecuencias de cada valor distinto.

Agregación global:

Si hacemos `df["poblacion"].sum()`, obtendremos la suma de la población de todas las regiones en el DataFrame. Podríamos calcular también densidad de población agregada, pero en este caso no tiene sentido sumar densidades. Más útil: promedio de poblaciones, etc.

Ejemplo:

```
pob_total = df["poblacion"].sum()
pob_media = df["poblacion"].mean()
print(f"Población total: {pob_total}")
print(f"Población promedio: {pob_media:.0f}")
```

Esto sumará las 6 poblaciones (de nuestro ejemplo, total ~11,860,000) y la media (~1,976,667). Formateamos la media sin decimales con `:.0f` en el f-string.

Si llamamos `df.sum()`, por defecto sumará por columnas numéricas retornando una Series con el total de cada columna numérica:

```
totales = df[["poblacion", "superficie"]].sum()
print(totales)
Salida:
poblacion    11860000
superficie    278896
dtype: int64
```

Nos indica que la suma de las poblaciones es 11,860,000 y la suma de las superficies es 278,896 km² (que sería la suma del área de esas 6 regiones; no representa algo útil en sí, pero es un número). Notemos que incluyo `df[["poblacion", "superficie"]].sum()` para asegurar que solo sume numéricas. `df.sum()` sin especificar incluiría también la columna "region", pero al ser no numérica, por defecto `sum()` en strings concatenaría las cadenas (en este caso concatenaría todas las regiones en una sola cadena larga, lo cual existe como comportamiento, pero raramente es lo deseado). Por eso típicamente limitamos a las columnas numéricas o usamos métodos en Series específicas.

Agregación condicional: Podemos combinar con filtros. Por ejemplo, calcular la población total solo de regiones por debajo de 1 millón:

```
pob_sub1m = df[df["poblacion"] < 1000000]["poblacion"].sum()
print("Población combinada de regiones < 1 millón:", pob_sub1m)
```

Esto filtra el DataFrame a filas donde población < 1e6 (Atacama y Aysén en nuestro caso) y luego suma la columna población. Sería 300000 + 110000 = 410000.

Conteos y valores únicos:

Podemos querer saber cuántas regiones hay (debería ser 6, trivial de df.shape también).

df["region"].count() contaría valores no nulos (6). Pero si tuviera nulos es distinto a len(df) que contaría filas inclusive con null.

df["region"].nunique() daría número de valores únicos (6 en este caso; útil si hay repeticiones).

df["region"].unique() daría la lista de nombres de región presentes.

value_counts es más útil en otros contextos, ej: si tuviésemos una columna "zona" que indica si es norte/centro/sur, podríamos contar cuántas regiones caen en cada zona:

```
# Esto es hipotético, asumiendo existiera df["zona"]
```

```
df["zona"].value_counts()
```

daría, por ejemplo: Sur 3, Centro 2, Norte 1 (si esa fuera la distribución, es un recuento por categoría).

Creación de nuevas columnas (derivadas): Aunque no estaba explícitamente en la lista, es muy común en manipulación de datos crear columnas basadas en otras (por ejemplo, densidad poblacional = población/superficie). Esto es una operación vectorizada simple:

```
df["densidad"] = df["poblacion"] / df["superficie"]
print(df[["region", "densidad"]])
```

Salida:

	region	densidad
0	Metropolitana	460.942599
1	Valparaíso	109.787289
2	Biobío	64.906878
3	Araucanía	31.388172
4	Atacama	4.007492
5	Aysén	1.013925

Hemos añadido la columna "densidad" al DataFrame (habitantes por km²). Vemos claramente la diferencia: Metropolitana ~461 hab/km², Aysén ~1 hab/km². Esta nueva columna se calculó elemento a elemento gracias a que pandas aplica la división a cada fila correspondiente.

Con la columna densidad, podríamos hacer agregaciones también, por ejemplo `df["densidad"].mean()` para densidad media de estas regiones (que no es muy interpretable, pero es posible).

Nota: La creación de columnas nuevas se suele considerar manipulación fundamental también, aunque no estaba listada explícitamente en los contenidos conceptuales, la hemos incluido como parte del análisis de datos inicial porque es muy común.

Imagen 5: creación de nueva columna en un dataframe

ÍNDICE	REGIÓN	POBLACIÓN	SUPERFICIE (KM ²)	DENSIDAD (HAB/KM ²)
0	Metropolitana	8,083,728	17,508	461.73
1	Valparaíso	1,960,178	16,396	119.56
2	Biobío	1,617,542	23,890	67.71
3	Araucanía	1,014,354	31,842	31.86
4	Aysén	107,316	108,495	0.99

Fuente: imagen creada con IA

7. Nociones de visualización básica de datos

La visualización de datos es un componente crucial de la ciencia de datos, pues facilita la interpretación de resultados y la comunicación de hallazgos. En esta sección veremos cómo generar gráficos simples utilizando Python. Nos enfocaremos en herramientas básicas:

- **matplotlib**: es la biblioteca estándar para visualización en Python, muy flexible y de bajo nivel (similar en estilo a MATLAB). Permite crear todo tipo de gráficas (líneas, barras, dispersión, histogramas, etc.).
- **pandas**: integra capacidades de graficar utilizando matplotlib por debajo, con métodos convenientes en DataFrames/Series para gráficos rápidos.

Nuestro objetivo es ilustrar cómo visualizar distribuciones o resultados de datos procesados, alineado con la noción de entregar salidas.

7.1 Importando matplotlib

Matplotlib típicamente se importa así:

```
import matplotlib.pyplot as plt
```

pyplot es un módulo que proporciona funciones para crear y manipular figuras de forma imperativa (inspirado en MATLAB). En entornos tipo Jupyter Notebook, se usa `%matplotlib inline` para mostrar gráficas dentro del notebook, pero asumiremos un entorno general.

7.2 Tipos de gráficos básicos

Algunos gráficos comunes y sus usos:

- **Gráfico de líneas (line plot)**: muestra la evolución de un valor continuo a lo largo de otra variable, típicamente tiempo. Ej: evolución de temperatura a lo largo de días.
- **Gráfico de barras (bar plot)**: compara valores discretos/categoricos. Ej: población por región, ventas por categoría.
- **Histograma**: muestra la distribución de frecuencias de un conjunto de valores numéricos (divididos en rangos). Ej: distribución de ingresos de personas.
- **Gráfico de dispersión (scatter plot)**: representa pares de valores (x, y) para ver correlaciones o distribución conjunta. Ej: edad vs ingreso de individuos, para ver si hay relación.
- **Pie chart (gráfico de torta)**: para proporciones de un total (aunque a veces menos recomendado en círculos analíticos, es popular para ilustrar porcentajes de composición).

Ejemplo: Gráfico de barras con matplotlib

Volviendo a nuestro ejemplo de las regiones chilenas, sería útil visualizar la población de cada región comparativamente. Un gráfico de barras es adecuado para esto (categorías en eje X, valor numérico en Y).

```
import matplotlib.pyplot as plt
# Datos a graficar (podemos extraer de nuestro DataFrame df)
regiones = df["region"]
poblaciones = df["poblacion"]

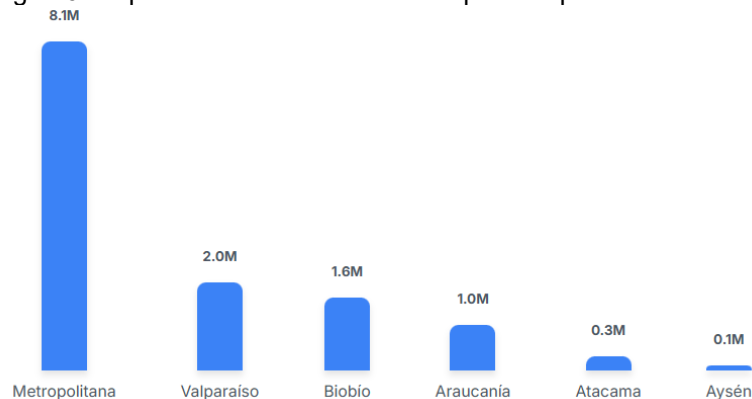
plt.figure(figsize=(8,5)) # opcional: tamaño de la figura en pulgadas ancho x alto
plt.bar(regiones, poblaciones, color='skyblue')
plt.title("Población por Región")
plt.xlabel("Región")
plt.ylabel("Población (habitantes)")
plt.xticks(rotation=45) # rotar las etiquetas del eje X 45° para que no se encimen si son largas
plt.tight_layout() # ajusta los márgenes para que las etiquetas que rotamos se vean completas
plt.show()
```

Desglose:

- `plt.figure(figsize=(8,5))` inicializa una figura de 8x5 pulgadas (esto puede ayudar en notebooks a hacerla más grande).
- `plt.bar(x, y)` crea un gráfico de barras con las categorías x (nombres de regiones) y valores y (poblaciones). Elegimos un color (skyblue).
- `plt.title`, `plt.xlabel`, `plt.ylabel` agregan título y etiquetas de ejes.
- `plt.xticks(rotation=45)` rota las etiquetas del eje X 45 grados para que, por ejemplo, "Metropolitana" y "Valparaíso" se lean verticalmente inclinadas y no se monten una sobre otra.
- `plt.tight_layout()` ajusta automáticamente los espacios para que todo el texto quepa dentro del lienzo.
- `plt.show()` muestra la gráfica (en algunos entornos como scripts no es estrictamente necesario, pero en notebooks sí para renderizarla).

A continuación, una imagen resultante del gráfico de barras con seis barras, donde claramente se ve la barra de "Metropolitana" mucho más alta que las demás, Aysén muy baja, etc. Esto ayudaría a visualizar la disparidad poblacional comentada.

Imagen 6: Representación visual de la disparidad poblacional en Chile



Fuente: Imagen creada con IA

Del gráfico, uno interpretaría rápidamente qué región tiene la mayor población y las diferencias relativas.

Ejemplo: Histograma con pandas

Si quisiéramos ver la distribución de las poblaciones, podríamos hacer un histograma. Con tan solo 6 datos no es muy significativo, pero supongamos un ejemplo con más datos, por ejemplo distribución de ingresos mensuales de 1000 personas. Pandas nos permite hacer:

Ejemplo imaginario: serie de ingresos en pesos (miles de pesos)

```
ingresos = pd.Series([500, 600, 1200, 450, 800, 400, 1500, ...]) # etc.
ingresos.plot(kind='hist', bins=10, title="Distribución de Ingresos")
plt.xlabel("Ingresos (miles de pesos)")
plt.show()
```

Aquí usamos Series.plot con kind='hist' para histograma, especificando 10 *bins* (intervalos). Pandas delega en matplotlib la creación, nosotros igualmente podemos ajustar título y etiquetas.

Ejemplo: Gráfico de líneas con datos temporales

Para dar otro ejemplo breve, imaginemos que tenemos los casos mensuales de influenza en Chile durante un año:

```
meses = ["Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov",
"Dic"]
casos = [120, 80, 200, 400, 800, 1500, 3000, 2500, 900, 300, 150, 100] # ficticio
plt.plot(meses, casos, marker='o')
plt.title("Casos mensuales de influenza (2025)")
plt.xlabel("Mes")
plt.ylabel("Número de casos")
plt.show()
```

Esto dibujaría una línea pasando por cada mes, con un marcador circular en cada mes (marker='o'), mostrando cómo los casos aumentan en invierno (peak en julio) y disminuyen luego, lo cual es coherente con patrones estacionales de influenza. Como se muestra en la siguiente imagen:

Imagen 7: ejemplo de casos mensuales de influenza



Fuente: imagen creada con IA con datos anteriormente expuestos.

Guardar o mostrar gráficos: Aquí usamos `plt.show()` para mostrarlos en pantalla. También es posible guardar la figura directamente a un archivo usando `plt.savefig("grafico.png")`. Esto es útil si queremos luego insertar la imagen en un informe.

Interactividad vs estático: En un notebook, `%matplotlib inline` hace que los gráficos aparezcan embebidos bajo la celda. En un script normal, `plt.show()` abre una ventana con la gráfica. En un entorno sin interfaz (por ejemplo, generando reportes automáticamente), usar `savefig` para guardar la imagen es útil.

7.3 Gráficos integrados con pandas

Como mencionamos, pandas tiene métodos `.plot()` que hacen más fácil generar gráficos estándar sin trabajar directamente con matplotlib.pyplot para cada detalle. Por ejemplo:

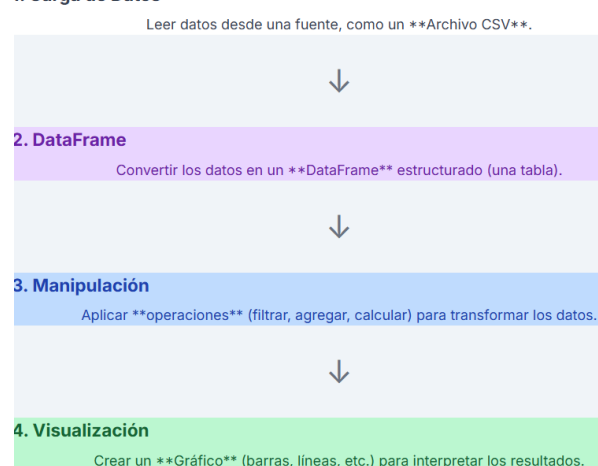
- `df.plot(kind='bar', x='region', y='poblacion')` generaría un gráfico de barras similar al anterior.
- `df.plot(kind='scatter', x='superficie', y='poblacion')` haría un scatter plot población vs superficie (veríamos la relación entre superficie y población de regiones; probablemente puntos dispersos, con Aysén siendo punto de gran superficie pero baja población).
- Para series: `df["poblacion"].plot(kind='hist')` un histograma de poblaciones.

Detrás de escena, pandas crea la figura usando matplotlib, y retorna un objeto Axes que podemos personalizar si quisiéramos (por ejemplo, añadir títulos usando `plt.title` después, o vía Axes methods).

Conclusión de visualización básica: Con solo pocas líneas podemos generar visualizaciones que complementan el análisis de datos. Esto cumple con la idea de presentar salidas interpretables (como gráficos) mencionada en los resultados de aprendizaje. Incluso un gráfico simple puede revelar patrones no obvios en tablas: por ejemplo, viendo el gráfico de barras de población, se aprecia visualmente la disparidad enorme entre la Región Metropolitana y las demás, y también que Aysén es minúscula en población comparada incluso con la penúltima. Un histograma podría mostrar distribución sesgada, etc.

Al producir informes o cuadernos ("notebooks") académicos, es común insertar estas visualizaciones junto con el código y explicaciones, para activar el aprendizaje visual del estudiante. Por eso, hemos indicado dónde podrían agregarse imágenes de gráficos, salidas de código, diagramas de estructuras, etc., a lo largo del documento. Esto ayudará a mantener la atención y a clarificar conceptos.

Imagen 8: Flujo de trabajo en ciencia de datos



Fuente: imagen creada con IA

8. Integrando conceptos en un script completo

Para finalizar esta unidad, integraremos varias de las operaciones aprendidas en un pequeño ejemplo práctico. El objetivo es desarrollar un programa en Python que:

- Cargue un conjunto de datos desde un archivo.
- Realice algunas operaciones de limpieza o transformación.
- Utilice estructuras de control y funciones para automatizar pasos repetitivos.
- Aplique operaciones con pandas (filtrado, ordenamiento, agregación).
- Produzca una salida interpretable, por ejemplo una tabla resumen o gráfico.

Escenario: Supongamos que contamos con un archivo `regiones.csv` (como el que usamos en ejemplos) con datos de varias regiones de Chile. Queremos:

1. Leer los datos.
2. Verificar si hay valores faltantes o poblaciones negativas (datos sucios) y limpiarlos.
3. Calcular la densidad poblacional de cada región (nueva columna).
4. Filtrar las 3 regiones más pobladas.
5. Guardar un resumen de resultados en un archivo nuevo y además generar un gráfico de barras de población vs densidad.

Este ejemplo es simplificado, pero toca varios puntos: lectura de archivo (I/O), uso de condiciones (por ejemplo, para limpieza), uso de pandas para manipulación, y generación de salidas (archivo y gráfico).

Empecemos paso a paso, documentando en el código:

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Cargar datos desde CSV a DataFrame
df = pd.read_csv("regiones.csv")
print("Datos originales cargados:")
print(df.head()) # mostrar primeras filas para verificar

# 2. Limpieza básica: verificar valores faltantes o inválidos
if df.isnull().any().any():
    # Si hay *cualquier* valor nulo en el DataFrame
    print("Se encontraron valores faltantes. Filtrando filas completas...")
    df = df.dropna() # eliminar filas con NA (alternativamente, podríamos llenar con algún valor)
# Verificar poblaciones negativas (no deberían existir)
if (df["poblacion"] < 0).any():
    print("Se encontraron poblaciones negativas, corrigiendo a valor absoluto.")
    df.loc[df["poblacion"] < 0, "poblacion"] = df["poblacion"].abs()

# 3. Crear nueva columna de densidad poblacional (hab/km^2)
df["densidad"] = df["poblacion"] / df["superficie"]

# 4. Filtrar las 3 regiones más pobladas
df_sorted = df.sort_values(by="poblacion", ascending=False)
top3 = df_sorted.head(3)
print("\nTop 3 regiones por población:")
```

```
print(top3[["region", "poblacion", "densidad"]]) # mostrar nombre, población y densidad de top3
```

```
# 5. Guardar resumen a archivo CSV
top3.to_csv("regiones_top3.csv", index=False)
print("\nArchivo 'regiones_top3.csv' generado con las 3 regiones más pobladas.")
```

```
# 6. Generar un gráfico comparativo de Población vs Densidad para todas las regiones
fig, ax1 = plt.subplots(figsize=(8,5))
```

```
color = 'tab:blue'
ax1.set_title("Población y Densidad por Región")
ax1.set_xlabel("Región")
ax1.set_ylabel("Población", color=color)
ax1.bar(df["region"], df["poblacion"], color=color, label="Población")
ax1.tick_params(axis='y', labelcolor=color)
plt.xticks(rotation=45)
```

```
# Crear segundo eje Y para densidad
ax2 = ax1.twinx() # segundo eje que comparte eje X
color = 'tab:green'
ax2.set_ylabel("Densidad (hab/km^2)", color=color)
ax2.plot(df["region"], df["densidad"], color=color, marker='o', label="Densidad")
ax2.tick_params(axis='y', labelcolor=color)
```

```
# Añadir leyendas
ax1.legend(loc="upper left")
ax2.legend(loc="upper right")
```

```
plt.tight_layout()
plt.show()
```

Explicuemos las partes importantes del script integrado:

- Lectura (paso 1): Leemos regiones.csv a DataFrame. Imprimimos head() para asegurarnos de que se cargó correctamente (esto es solo en la fase de desarrollo/depuración; en un script final tal vez no imprimimos nada o solo un mensaje de éxito).
- Limpieza (paso 2): Usamos df.isnull().any().any() para chequear *si existe al menos un valor nulo* en todo el DataFrame (el primer .any() aplicaría por columna, el segundo sobre todo el DataFrame). Si hay, informamos y usamos dropna() para eliminar filas incompletas. Alternativamente podríamos fillna() para rellenar, pero al no tener especificaciones, optamos por eliminar las incompletas.

Luego chequeamos si alguna población es negativa ((df["poblacion"] < 0).any()). Si encontramos, convertimos esos valores a positivos usando .abs(). Esto es un ejemplo de corrección de datos atípicos; en datos reales podríamos investigar por qué hay negativo, pero para propósitos de la tarea, mostramos uso de if + loc para corrección.

- Nueva columna densidad (paso 3): Simple vectorized operation as seen: hab/km².
- Filtrar top 3 (paso 4): Ordenamos descendientemente por población y tomamos head(3). Luego imprimimos el resultado con solo las columnas de interés. Esto demuestra ordenamiento + selección de subset.

- Guardar CSV (paso 5): Usamos `to_csv` para guardar el DataFrame `top3` a un archivo 'regiones_top3.csv', sin incluir el índice (ya que no es relevante en este reporte). Esto ilustra salida de datos a archivo después de manipularlos.
- Graficar (paso 6): Aquí hicimos algo un poco más complejo: un gráfico con dos ejes Y.
 - Primero creamos un gráfico de barras de población (eje Y izquierdo, color azul).
 - Luego creamos un eje gemelo `ax2` para densidad (eje Y derecho, color verde) y graficamos una línea con marcador para densidad.
 - Esto nos permite ver en un mismo gráfico la comparación: por ejemplo, Metropolitana tiene la barra más alta (población) y también un punto de densidad muy alto. Regiones como Aysén tendrán barra pequeña y punto de densidad muy bajo.

Pusimos leyendas en esquinas opuestas para identificar azul = población, verde = densidad. Rotamos etiquetas X por legibilidad.

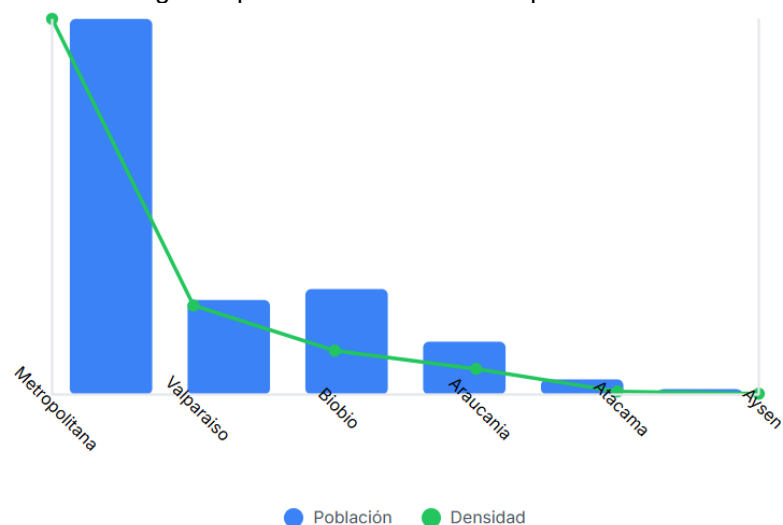
Este gráfico es quizás avanzado para un primer ejercicio, pero sirve para *visualizar dos medidas diferentes en escala distinta*. Nos asegura que generamos una salida gráfica interpretando los datos procesados, completando así la integración.

Ejecutando este script paso a paso:

- Tras lectura, imaginemos que `regiones.csv` no tenía nulos ni negativos, entonces se saltan las correcciones (no se imprimirá nada en esos `if`).
- Crea densidad.
- Top 3 pobladas serían: Metropolitana, Valparaíso, Biobío (como vimos). `regiones_top3.csv` contendrá esas filas.
- El gráfico de salida mostrará 6 barras azules y 6 puntos verdes con línea, etiquetas para población y densidad.

Un posible resultado sería como el que se muestra a continuación:

Imagen 9: posible resultado del script anterior.



Fuente: imagen creada con IA basándose en el código anteriormente expuesto.

Preguntas de activación del aprendizaje:



COMPRUEBO MI APRENDIZAJE

1. ¿Qué diferencia hay entre una lista y una tupla? Da un ejemplo concreto aplicado a datos de regiones de Chile.
2. Escribe un código en Python que lea un archivo ventas.csv con dos columnas: producto y cantidad. El programa debe calcular el total vendido y mostrarlo.
3. ¿Por qué es importante garantizar la coherencia y consistencia de los datos? Da un ejemplo aplicado a información pública chilena.

En el siguiente QR puedes revisar las respuestas esperadas

**Información complementaria para el aprendizaje:****Información complementaria**

Link 1: Tipos integrados de Python (documentación oficial en español)

https://docs.python.org/es/3/library/stdtypes.html?utm_source=chatgpt.com

Link 2: 10 minutos para pandas” (pandas 2.3.2) — tutorial rápido

https://pandas.pydata.org/docs/user_guide/10min.html?utm_source=chatgpt.com

Link 3: Matplotlib: documentación oficial

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html?utm_source=chatgpt.com

Conclusión

En esta unidad hemos cubierto los fundamentos de programación aplicados a la manipulación de datos en Python:

- Conocimos los tipos básicos y estructuras nativas de Python (listas, tuplas, diccionarios) para representar datos en memoria.
- Aprendimos a controlar el flujo de ejecución mediante condicionales y bucles, lo cual es esencial para aplicar reglas de limpieza y repetir procesos en conjuntos de datos.
- Vimos cómo encapsular lógica reutilizable en funciones, modulando nuestras soluciones.
- Practicamos la interacción con archivos para leer datos de fuentes externas (por ejemplo CSV) y escribir resultados (reportes o datos procesados).
- Introdujimos dos librerías clave, NumPy y pandas, que nos permiten manejar datos de forma eficiente y conveniente. En particular, pandas nos facilitó cargar datasets, explorarlos rápidamente y realizar operaciones como filtros, ordenamientos y agregaciones con mínima codificación manual.
- Exploramos la creación de visualizaciones básicas, que resultan cruciales para interpretar los datos y comunicar hallazgos. Gráficos de barras, líneas, histogramas, entre otros, nos dieron un vistazo visual a la información.
- Finalmente, integramos todo en un ejemplo práctico, evidenciando cómo un pequeño script puede reunir múltiples operaciones (lectura, limpieza, transformación, análisis, visualización) para producir resultados valiosos, cumpliendo así con todos los indicadores de logro de esta unidad (desde ejecutar programas básicos de datos hasta desarrollar scripts integrales con salidas interpretables).

Referencia Bibliografía

López, A., & Rojas, A. L. (2021). Introducción a Python para estudiantes de ciencias. Editorial de la Universidad Pedagógica y Tecnológica de Colombia (UPTC). <https://ipss.odilo.us/info/introduccion-a-python-para-estudiantes-de-ciencias-02608297>

Kelleher, J. D., & Tierney, B. (2021). Ciencia de datos (Serie de conocimientos esenciales de MIT Press). Ediciones UC. <https://ipss.odilo.us/info/ciencia-de-datos-la-serie-de-conocimientos-esenciales-de-mit-press-02574729>

Referencia del presente documento:

Instituto San Sebastián, Dirección de Innovación Académica (2025). *Apunte 1: Fundamentos de programación y manipulación de datos*.